Provisioning Edatec IPC 3100 using nyle imager.

Controls Group Nyle

January 26, 2026

Abstract

Its more or less as you would expect, but there are some CM5 compatibility issues to beware of. The repo versions of RPI boot are not new enough to encompass CM5 hardware, and must be built from source. until some handsome and noble gentleman takes a minute to back-port the newer source code into Debian compatible packages, or bugs the Raspberry pi organization to release updated code in his free time...

Well anyway, there may be other software to do the same process more reliably, with default supporting packages. Here we are trying to adhere to the official documentation as much as possible.

Official Docs

`https://edatec.cn/docs/an/an19-installation-of-ubuntu-os-on-ed-ipc/#_2-3-flashing-to-emmc`

The documentation is good, with pictures, but is written from the windows user perspective, and doesn't seem to be aware of the CM5 chip-set issue. This compatibility issue may exist on Windows as well, as it is a firmware compatibility issue, and an official release has not been made in over 4 years

Unofficial documents

usbboot
`https://github.com/raspberrypi/usbboot`

Raspberry Pi USB device boot - RPIBOOT mentioned in the official docs had its last official release in 2022, but the CM5 came out in 2024. While the source code has been added to fix this issue, newer releases have not been created. .. which means this issue is probably present on any OS.

Creating A Disk Image using nyle-imager

System Requirements for nyle-imager

The nyle-imager requires the user to be online durring its operation, and using a debian based Linux distribution. Docker, qemu, packages are required to run.

Using nyle-imager

nyle-imager is a Nyle internal application which builds a MCPLA bootable raspberry image based on a docker configured MPCLA. The software required to build the image is pulled from a private repository in the Nyle system, and built to contain all the software required to garner a fully functional MCPLA after minimal post install configuration. A variety of json files dictate the package installs to the output image, following the same Debian based command style you may be familiar with. from the nyle-imager source directory:

```
$ bash build-mcpla.sh
```

About the nyle-imager build process

During the build process, you may see red highlighted text, indicating some package is unable to initialize. This is normal, as the installation is not complete, packages and process are not expected to be fully instantiated. Some packages may be deprecated, which will give a warning, but this is not a critical fault.

```
.2-1 [77.5 kB]
    arm: Get:98 http://deb.debian.org/debian bookworm/main arm64 vim-runtime all
 2:9.0.1378-2+deb12u2 [7027 kB]
    arm: Get:99 http://deb.debian.org/debian bookworm/main arm64 vim arm64 2:9.0
.1378-2+deb12u2 [1428 kB]
==> arm: debconf: unable to initialize frontend: Dialog
==> arm: debconf: (TERM is not set, so the dialog frontend is not usable.)
==> arm: debconf: falling back to frontend: Readline
==> arm: debconf: unable to initialize frontend: Readline
==> arm: debconf: (This frontend requires a controlling tty.)
==> arm: debconf: falling back to frontend: Teletype
==> arm: dpkg-preconfigure: unable to re-open stdin:
    arm: Fetched 52.8 MB in 4s (11.8 MB/s)
    arm: Selecting previously unselected package dnsmasq.
    arm: (Reading database ... 56212 files and directories currently installed.)
    arm: Preparing to unpack .../00-dnsmasq_2.90-4~deb12u1_all.deb ...
```

Figure 1: Process information during run time.

Editing the nyle-imager parameters

The nyle-imager is a set of build related scripts, with controls over how the image is built and software to be installed. Notably for the MCPLA project, mcp-la-2.sh installs a specific version of the MCPLA from the NyleWaterHeating GitHub using the parameters:

```
branch=1.007-review
ui_branch=feature/mcp-serial-number
```

Changing these variables will change the version of the MCPLA software installed to the disk image.

In the mcp-la-1.sh script, the ‘apt-get install‘ parameter may be used to install standard packages in the Raspberry Pi linux universe.

```
apt-get install -y gpg mosquitto-clients sudo wget curl git \
jq mosquitto mosquitto-clients dnsmasq udisks2 udiskie \
vim tmux mc inotify-tools
```

When you are ready, run:

```
$ bash build-mcpla.sh
```

Upon successful completion you will see:

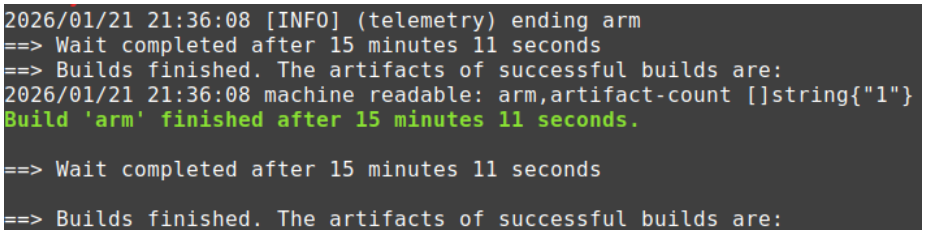

Figure 2: Success!

Troubleshooting nyle-imager

The nyle-imager builds a complete bootable disk image based on configured parameters. Some of those parameters come from online sources. If the sources are unavailable, possibly due to mis-specification, or a lack of connectivity, the build process will fail. Important details about the source of the failures will be in the terminal window after running

```
$ bash build-mcpla.sh
```

Installing RPIBOOT from source

What RPIBOOT does:
mounts your CM5 IPC as a standard disk object, making your install a simple input file to output device operation.

your distribution specific RPIBOOT may not fail, it might be fine for versions not using the CM5, or older hardware. The main problem with RPIBOOT is that the incompatible source tree will never fail, it goes into an infinite loop with a message that states looking for BCM 2711... which you might not know is an older broadcom chip. The CM5 uses a Broadcom 2712.

but it never moves on from here. Install pkg dependencies:

```
$ sudo apt-get install -y git libusb-1.0-0-dev pkg-config build-essential
```

Get the source:

```
$ git clone https://github.com/raspberrypi/usbboot
$ cd usbboot/

$ make
$ sudo make install
```

Putting it all together

Take your powered down Edatec IPC3100, and connect power. You may opt to have a screen attached, or have RPIBOOT ready and waiting, but the important step is to put the unit in boot mode by pressing the reset button located on the same side as the USB ports. Pressing the reset button puts the IPC3100 into boot mode, and exposes the device as a writeable disk. As per the manufacturers original instructions, you will need the micro USB cable inserted into the device, and removing the read DIN rail mounting hardware may be required.

```
$ sudo rpiboot -v
```

List relevant info:

```
$ lsblk -o NAME,SIZE,MODEL,SERIAL,TRAN
```

Unmount your CM5 to write directly to the disk

```
$ sudo umount /dev/sda1 2>/dev/null
$ sudo umount /dev/sda2 2>/dev/null

$ lsblk -f /dev/sda
```

Clone your MCPLA disk image created with the nyle-imager software to the applicable device listed in previous steps. You should see anew file name mcpla-reapios-lite-arm64.img in your nyle-imager root directory. From there, run:

```
$ sudo dd if=mcpla-reapios-lite-arm64.img of=/dev/sda bs=4M conv=fsync status=progress
```

Synchronize the write cache to the disk. .. conv=fsync should do this, but sometimes dd is funny.

```
$ sync
```

Verify disk operations are complete, if lsof says nothing, no thing is accessing the CM5 boot media, and the device may be safely removed.

```
$ lsof /dev/sda
```

Command to power down a disk (CM5, Edatec IPC3100) using udisksctl

```
$ udisksctl power-off  -b /dev/sda
```

Final bits of installation

Now that the image is on the device, disconnect the micro USB cable, and press the reset button. The disk image is now installed on your Raspberry Pi, and the installation process is nearly complete. Upon first boot, the disk image will expand from 8GB to whatever the available disk storage area is. Other processes will also occur such as configuring ssh keys. You may have to Configure keyboard layout, and configuring the default user. Follow the onscreen prompts to complete your installation.

Boring Process details

heres the difference between NOT running RPIBOOT, and running RPIBOOT

```
$ lsblk -o NAME,SIZE,MODEL,SERIAL,TRAN
NAME          SIZE MODEL                     SERIAL          TRAN
nvme0n1     931.5G Samsung SSD 990 PRO 1TB S73VNU0XC00913E nvme
|-- nvme0n1p1   512M                                       nvme
`-- nvme0n1p2   931G                                       nvme

$ sudo rpiboot
[sudo] password for eri:
RPIBOOT: build-date 2026/01/20 pkg-version local d6d87604
...

Waiting for BCM2835/6/7/2711/2712...
...
Second stage boot server done


$ lsblk -o NAME,SIZE,MODEL,SERIAL,TRAN
NAME          SIZE MODEL                                    SERIAL           TRAN
sda          29.1G Raspberry Pi multi-function USB device 919bf90242ca38f9 usb
|-- sda1        512M
`-- sda2       28.6G
nvme0n1     931.5G Samsung SSD 990 PRO 1TB                S73VNU0XC00913E  nvme
|-- nvme0n1p1   512M                                       nvme
`-- nvme0n1p2   931G                                       nvme
```

Exit Boot Mode

If you have inadvertently entered boot mode, unplug the micro USB cable, and press the reset button to reboot.

You may observe Edatec-IPC3100 has entered boot mode with the HDMI plugged in, the screen may read:

```
Raspberry Pi 64-bit Mass Storage Gadget
rpimsg login: root (automatic login)
.....
host plug operations halted.
```

where it will then wait for whatever operation to transpire.

Pulling the power could work as well, but in this test we have the model with the Super-Caps present, and pulling the power might take over 5 minutes to fully loose power.

Unit Testing Methodology

Testing includes a hardware inspection and load testing.
Visual inspection was satisfactory, no issues in quality, cleanliness or construction were observed.

Software used:

lm-sensors for collection of thermal sensor data
stress-ng to do load testing.

```
$ sudo apt install lm-sensors
$ sudo sensor-detect --auto

$ sudo apt get install stress-ng
$ stress-ng --cpu 0 --cpu-method all --timeout 0
```

A bash script was used to track time, heat, load, a frequency, to monitor for thermal cut out, load dropping, time, and generating a csv for making a chart.

We ran the system with our MCPLA code while stressing the computer with a bash script to log thermals over time. Artificially maxing out hardware use for an extended period of time.

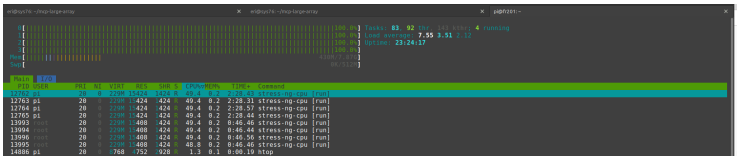

Figure 3: stress loading CPU.

The thermal effect of 100% load over time:

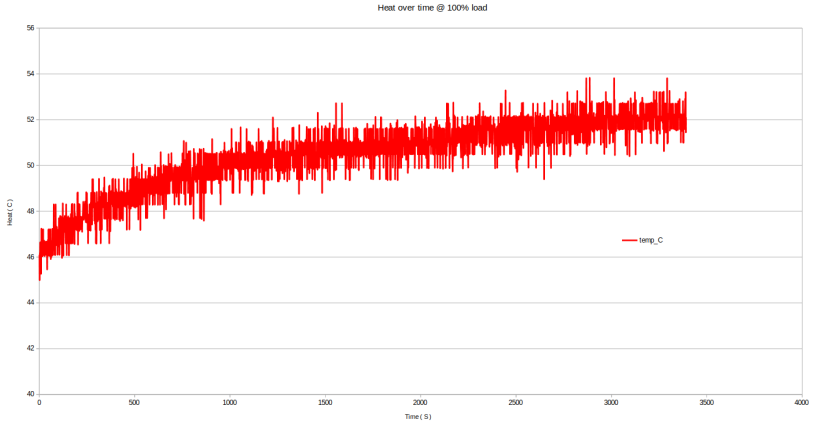

Figure 4: Thermals over time.

Brief analysis of IPC 3100 performance

We tested the unit at 100% load for 4 hours in the MCPLA enclosure. The maximum CPU operational temperature reached was 56C. No interruptions to the MCPLA Process were observed during this time. Small response delays were noticeable while running shell access operations, which is expected. At no point did the system drop any connections, loose IO interfaces, crash, or become unresponsive. Peak observed power consumption was 8 watts. the IPC 3100 list max power is 24 Watts. Max system wide power consumption observed was 2.6 amps at 12 VDC or 31.2 Watts.